

答辩问题回答

一、选题与概念界定

Q1：为什么必须使用深度学习而不是 GLM 或 XGBoost？

本课题选择 MLP 并非主张"深度学习必然优于 XGBoost"，而是基于以下考量：

- 研究定位**：题目明确为"基于深度学习的车险理赔预测系统"，研究目标之一就是探索 MLP 在该场景下的可行性与优化空间，这本身构成一个完整的研究问题。XGBoost 等树模型在表格数据上确实有竞争力，但若直接用 XGBoost 训练一个分类器并接入系统，模型部分的深度和可调空间会大幅缩减，难以支撑一章论文的工作量。
- 灵活性优势**：MLP 的模块化结构（残差块、归一化层、激活函数、分类头等组件可独立替换和组合）使得消融实验能够系统地量化各组件的贡献——这在树模型中较难做到同等粒度的对照分析。
- 可扩展性**：MLP 架构为后续引入多任务学习（同时预测理赔发生与理赔金额）、特征交互层（如 Cross Network）、嵌入层（处理高基数类别变量）等扩展预留了空间，而 XGBoost 在架构扩展上的自由度较为有限。
- 工程整合**：MLP 基于 PyTorch，与 FastAPI 推理服务的整合生态更加自然，且支持 GPU 推理加速。但这不构成学术层面的核心理由。

坦率地说，缺少 XGBoost/GLM 基线对比是本文实验设计的一个不足，在"未来展望"或"系统缺陷"部分应当承认这一点。

二、模型结构与选择设计

Q2：浅层网络中残差连接的必要性？

诚然，残差连接最初为极深网络设计。但在本模型的 4 残差块（8 层全连接）场景下使用残差连接，理由如下：

- 消融实验中有对照组**：组件消融实验中设置了 `use_residual=False` 的对照组（RunPaperAblations.py 中 `component_no_residual`），将残差块替换为同等宽度的普通前馈块。实验结果显示残差连接带来的 F1 增益约为 0.005-0.008，AUC 增益约为 0.001-0.002——幅度确实有限，但方向一致为正。
- 作用机制不同**：在浅层网络中，残差连接的主要价值不是"缓解梯度消失"，而是让网络更容易学习恒等映射——即当增加层数不带来收益时，网络可以通过 shortcut 近似"跳过"多余层，避免深度增加反而损害性能。
- 维度切换时的平滑过渡**：沙漏型结构中存在 128→256 和 256→128 的宽度变化，残差连接的线性投影 shortcut 能确保维度切换时的信息传递不会出现断裂，比直接堆叠全连接层更稳定。
- 创新点的定位**：论文将"残差 MLP + 沙漏结构"并列为一个创新点，核心创新在于"两者的组合设计"而非单独的残差连接——即用残差块作为基本构建单元来实现沙漏型宽度变化，这不同于传统 MLP 的逐层等宽堆叠方式。

Q3：为什么选择 LayerNorm 而非 BatchNorm？

1. **对 batch size 不敏感**：BatchNorm 依赖 mini-batch 内的统计量（均值和方差），当 batch size 较小或不稳定时，batch 统计量的估计噪声较大。本模型默认 batch_size=128，不算特别小，但 LayerNorm 的计算完全独立于 batch 维度，训练和推理行为完全一致，不需要维护全局移动平均。
2. **与残差结构的配合**：LayerNorm 在 Transformer 架构中与残差连接的组合已被广泛验证有效。本模型的残差块设计（Linear → LayerNorm → GELU → Dropout → Linear → LayerNorm → +shortcut → GELU）借鉴了 Pre-LN Transformer 的思路——将归一化放在子层之前而非之后，有助于稳定训练初期的梯度传播。
3. **表格数据的特征异构性**：BatchNorm 对同一特征维度跨样本归一化，隐含假设不同样本的同一特征应当服从相近的分布。但在保险表格数据中，不同保单的同一特征（如年净保费）可能跨越多个数量级，跨样本的均值和方差未必是最合适的归一化基准。LayerNorm 对单个样本的所有特征维度归一化，更好地保留了样本间的相对差异。
4. **实际对比**：组件消融实验中设置了 `norm_type="none"` 的对照组（完全移除 LayerNorm），观察到性能明显下降，验证了归一化层的必要性。但确实没有设置 BatchNorm vs LayerNorm 的直接对照实验，这是消融实验覆盖面的一个不足。

Q4：为什么选择 GELU 而非 ReLU？

1. **平滑性优势**：GELU 在全体实数上处处可导，而 ReLU 在 $x=0$ 处不可导。对于使用了 LayerNorm + 残差连接的网络，GELU 的平滑梯度流有助于减少训练过程中的数值震荡。
2. **消融实验中有对比**：组件消融实验中设置了 `activation="relu"` 的对照组（`component_relu_activation`），将 GELU 替换为 ReLU。实验结果显示 GELU 相比 ReLU 在 F1 上有约 0.002-0.004 的微幅提升。差异不大，但考虑到推理阶段 GELU 的计算开销在 128-256 宽度的全连接网络中几乎可以忽略不计（单次推理的 GELU 计算量远小于矩阵乘法的计算量），选择 GELU 是合理的。
3. **"神经元死亡"问题**：ReLU 在输入为负时梯度为零，可能导致部分神经元永久失活。虽然在全连接网络中该问题不如 CNN 严重，但 GELU 在负半轴保留了一个非零的梯度，避免了这一隐患。

Q5：沙漏型结构的设计依据？

1. **设计思路而非严格数学推导**：沙漏型结构的设计参考了 autoencoder 的"编码-瓶颈-解码"思路——先升维扩展表示空间（128→256），在高维空间进行深化提取（256→256），再降维压缩去噪（256→128），最后精炼（128→128）。其核心思想是：让网络先在冗余的高维空间中学习特征交互，再将有用信息压缩到紧凑表示中。
2. **宽度变化的实验验证**：网络结构消融实验包含了不同宽度配置的对比——3 层小宽度（128,256,128）、3 层大宽度（256,512,256）、4 层小宽度（128,256,128,64）、4 层大宽度（256,512,256,128）、5 层小宽度（128,256,256,128,64）、5 层大宽度（256,512,512,256,128）。其中 5 层大宽度的沙漏型取得了最优结果。
3. **但确实缺少宽度模式对照**：实验中没有设置"等宽型""持续扩张型""瓶颈型"等不同宽度变化模式的直接对比。这是当前消融实验的一个局限——仅对比了不同深度和不同尺度的沙漏结构，但没有对比不同形态的结构。这在答辩时可以坦承，并说明由于实验时间和计算资源限制，优先验证了深度和宽度尺度的影响。
4. **与 autoencoder 瓶颈的关系**：经典 autoencoder 的沙漏结构中间有一个极窄的瓶颈层来强制信息压缩。本模型的沙漏结构没有极窄瓶颈（最窄 128 维，输入约 28-29 维，反而更宽），因此它更准确的描述应该是"先扩后缩的纺锤形结构"而非严格的沙漏。这个名称确实有不够精确之处。

Q6: Dropout 的设置依据?

1. **数值来源**: 主干 Dropout=0.25 和分类头 Dropout=0.15 是参考表格数据深度学习文献中的常见取值范围 (0.1-0.5) 并结合初步实验确定的, 不是通过系统化的网格搜索得到的最优值。
2. **位置设计**: 残差块内部的 Dropout 位置在第一层全连接 + LayerNorm + GELU 之后、第二层全连接之前。这种设计是在残差分支内部而非 shortcut 分支上施加 Dropout——这样做的意图是让 Dropout 作用于"需要学习的残差部分"而非"恒等传递的 shortcut 部分", 避免 Dropout 干扰 shortcut 的信息直通。
3. **不足之处**: 确实没有对不同 Dropout 比例 (如 0.1、0.2、0.3、0.4、0.5) 做系统性的网格搜索消融实验, 0.25 和 0.15 这两个值有一定经验成分。这是训练超参数调优方面的一个不足。

Q7: head_samples=10 在推理阶段还有意义吗?

1. **推理阶段确实意义有限**: 在 `model.eval()` 模式下, Dropout 被关闭, 分类头的每次前向计算完全确定。此时 `head_samples=10` 的 10 次前向计算结果完全相同, 取平均不改变最终输出。所以该设计只在训练阶段有意义。
2. **训练阶段的作用**: 在训练时 Dropout 处于激活状态, 每次前向传播随机丢弃一部分神经元 (概率 0.15)。对同一共享表示执行 10 次前向计算并对 logit 取平均, 等效于对这 10 次采样的预测进行集成——这是一种近似的 Monte Carlo Dropout 集成。
3. **与 MC Dropout 的区别**: 标准 MC Dropout 在推理阶段也保持 Dropout 开启, 通过多次前向传播的方差来估计模型的不确定性。本模型的 `head_samples` 在推理阶段关闭了 Dropout, 因此它不能提供不确定性估计——它只是一个训练时的正则化技巧, 不能算作完整意义上的集成学习或 MC Dropout。
4. **代码实际行为**: 查看 `Model.py` 第 132-137 行, `forward_logits` 在 `head_samples > 1` 时会循环调用 `self.clf_head(shared)` 多次并取平均。在 `model.eval()` 模式下, 由于 `self.clf_head` 内部的 Dropout 被禁用, 每次调用的结果完全相同, 取平均无意义。在 `model.train()` 模式下, 每次调用因 Dropout 随机丢弃不同神经元而产生不同输出, 取平均可以起到平滑效果。

Q8: 模型输出的概率是否经过校准?

未经过显式校准。 模型的输出概率直接来自 `sigmoid(logit)`, 没有使用 Platt Scaling、Isotonic Regression 或温度缩放等校准方法。

这意味着输出的概率值不能严格解释为"真实理赔概率"——例如, 模型输出 0.8 并不意味着该保单的真实理赔发生率就是 80%。不过, 由于 BCEWithLogitsLoss 本身是一个 proper scoring rule (适当评分规则), 在大量数据和充分训练的条件下, 输出的概率具有一定的排序一致性和近似校准性。此外, 最终业务判定使用的是自动搜索得到的分类阈值而非原始概率值本身, 因此概率校准的偏差对实际决策的影响有限。论文和系统文档中应明确说明概率值未经校准, 不应直接作为精算意义上的理赔概率使用。

三、特征工程与数据处理 (用户编号为"四")

Q9: 理赔相关字段的数据泄漏问题?

数据集中与理赔相关的字段及其处理如下:

- **N_claims_year (本年度理赔次数)**: 作为目标变量, 被排除在输入特征之外 (`DROP_COLS` 中包含 `TARGET_COLUMN`)。

- **Cost_claims_year (本年度理赔成本)**：被排除在输入特征之外（`DROP_COLS` 中包含 `AUXILIARY_LABEL_COLUMNS`）。该字段与目标变量 `N_claims_year` 高度相关——有理赔才有成本——若作为特征输入会造成严重的数据泄漏（模型直接通过成本 > 0 推断有理赔）。
- **N_claims_history (历史理赔次数)**：作为特征保留，因为它反映的是当前保单年度之前的历史理赔情况，属于预测时可获得的先验信息。
- **R_Claims_history (历史出险率)**：作为特征保留，同样属于历史信息。

在 `DataLoader.py` 第 29 行明确定义了 `DROP_COLS = ["ID", "Date_lapse", *AUXILIARY_LABEL_COLUMNS, TARGET_COLUMN]`，其中 `AUXILIARY_LABEL_COLUMNS = ["Cost_claims_year"]`，确保本年度理赔成本不会进入特征矩阵。

关于 `R_Claims_history` 的具体计算口径——其统计截止时间是否严格在预测时点之前——取决于原始数据集的定义。Segura-Gisbert 等的数据集文档中该字段被描述为 "historical claim rate"，论文假定其为历史信息。但严格来说，若该字段包含了当前年度的信息，则存在数据泄漏风险。这是一个值得在论文中更详细讨论的点。

Q10：最终特征维度不确定的问题？

维度不确定的原因是 `build_dataloaders()` 函数在运行时动态确定 `input_dim = x_train.shape[1]`，而 `x_train` 的列数取决于数据预处理后实际保留的特征列数。在论文写作时，因不同实验配置下（如是否包含某些可选特征）维度可能在 28 到 32 之间浮动，因此使用了 "约 28~32" 的表述。

在当前代码（`DataLoader.py`）中，`RAW_FEATURE_COLS` 定义了 26 个原始特征，经过日期特征工程后：删除 5 个日期列，新增 4 个日期差值列（`_days`）和 4 个派生特征列（`driving_experience_years`, `insured_age_years`, `contract_duration_years`, `vehicle_age_years`），再删除 `DROP_COLS` 中的 `ID`、`Date_lapse`、`Cost_claims_year` 和 `N_claims_year` 共 4 列。最终维度 = $26 - 5 + 8 - 4 = 25$ 列，加上编码和填充后的特征列数应为 28 列。论文应提供精确的特征列表（如表 4.3），如果用 "28~32" 的模糊表述，确实会影响可复现性。在实际代码运行中该数值是确定的，表格 4.3 中应已列出完整特征清单。

Q11：Type_fuel 二值编码的扩展性问题？

当前仅有两个燃料类型（汽油 P / 柴油 D），二值编码 0/1 与独热编码完全等价且在数值上更简洁。如果未来出现第三种及以上燃料类型（电动、混动、氢能等），二值编码将不再适用。

扩展方案：届时可以将 `Type_fuel` 改用独热编码或嵌入编码。由于该改动涉及特征维度的变化，需要同步更新 `RAW_FEATURE_COLS`、特征工程流程和模型的 `input_dim`，并重新训练模型。这就是为什么系统需要保留模型版本管理和 `scaler/preprocess_reference` 绑定机制。

为什么不一开始就用独热编码：对于当前仅有两个类别的场景，独热编码会引入一个冗余维度（两列完全共线），而二值编码更简洁且不引入冗余。工程上的原则是 "不为尚未发生的需求过度设计"。

Q12：类别变量的编码方式？

数据中的类别变量编码处理如下：

- **Type_fuel**：二值编码（P→0, D→1），如上所述。
- **Distribution_channel**（0=代理人, 1=保险经纪）、**Payment**（0=年缴, 1=半年缴）、**Second_driver**（0=无, 1=有）：原始数据中已为 0/1 二值整数，直接作为数值输入。
- **Type_risk**（1=摩托车, 2=货车, 3=乘用车, 4=农用车）、**Area**（0=农村, 1=城市）：原始数据中为整数编码，直接作为数值输入。

这些类别变量的编码方式是**标签编码（整数编码）**。对于无序类别变量（如 Type_risk 的四种车辆类型之间没有天然的数值大小关系），直接使用整数编码可能让模型误以为“农用车(4) > 乘用车(3) > 货车(2) > 摩托车(1)”——即引入了虚假的序关系。这是当前实现的一个局限。更合理的做法是对 Type_risk 使用独热编码或嵌入编码，但由于当前数据集中 Type_risk 仅有 4 个类别且分布不均（乘用车占绝大多数），整数编码的实际负面影响可能有限。论文中应讨论这一设计选择的局限性。

对于二值类别变量（Distribution_channel, Payment, Second_driver, Area），标签编码与独热编码等价，不会引入虚假序关系。

四、损失函数与训练策略（用户编号为“五”）

Q13：标签平滑与 pos_weight 是否相互抵消？

两者作用于损失函数的不同方面，不构成相互抵消，但它们之间确实存在微妙的交互：

1. **pos_weight** 直接放大正类样本的损失项——当 $y=1$ 时， $\text{loss} = \text{pos_weight} * [-\log(p)]$ 。它解决的是“模型对正类关注不够”的问题。
2. **标签平滑** 修改的是目标值本身—— $y=1$ 变为 $y_{\text{smooth}}=0.975$ ， $y=0$ 变为 $y_{\text{smooth}}=0.025$ 。这意味着 BCE 中的两项都始终存在非零贡献：即使真实标签为 1，也会有 0.025 的“负类惩罚”成分。
3. **交互分析**：对于正类样本（ $y=1$ ），平滑后的损失包含 $\text{pos_weight} * 0.975 * (-\log(p)) + \text{pos_weight} * 0.025 * (-\log(1-p))$ 两项——第二项是一个额外的正则化项，轻微惩罚模型对正类样本的过度自信。对于负类样本（ $y=0$ ），平滑后的损失为 $0.025 * (-\log(p)) + 0.975 * (-\log(1-p))$ ——第一项是一个轻微的正类惩罚。
4. **定性结论**：标签平滑对正类引入了约 2.5% 的“反向惩罚”，这在 $\text{pos_weight}=3.10$ 的放大下约等于 0.0775 的等效权重，相比正类主损失的 $3.10*0.975\approx 3.02$ 而言非常小。因此标签平滑不会实质性地抵消 pos_weight 的效果，两者的叠加是合理的。

Q14：为什么用 AUC 而非 F-beta 作为早停依据？

选择 AUC 作为早停监控指标而用 F-beta 作为阈值搜索目标，这两个选择服务于不同的目的：

1. **AUC 监控的是模型的排序能力**——即模型在没有选定阈值时的内在质量。Early Stopping 的目的是“在模型泛化能力最强的时刻保存参数”，而泛化能力最核心的体现就是排序能力。AUC 不受分类阈值影响，每个 epoch 的 AUC 可比性强。
 2. **F-beta 监控的是特定阈值下的分类性能**——而最优阈值每个 epoch 都在变化（自动阈值搜索的结果不同）。如果在 epoch A 时最优阈值为 0.72、F-beta 为 0.78，epoch B 时最优阈值为 0.68、F-beta 为 0.77——这个 F-beta 的下降可能来自阈值搜索的随机波动，而非模型排序能力的退化。因此直接用 F-beta 做早停可能引入阈值波动噪声。
 3. **AUC 和 F-beta 谁更贴合业务**：业务场景既关心排序（谁能被正确排序到高风险），也关心分类（最终谁被标记为高风险）。但排序是分类的基础——只有排序做好了，阈值搜索才能找到好的分类。因此早停用 AUC 保证排序质量，最终用 F-beta 选择操作阈值，两者各司其职。
 4. **如果 AUC 最优和 F-beta 最优的 epoch 不一致**：当前代码保存的是 AUC 最优的模型。因为 AUC 反映的是模型参数的泛化质量，而阈值搜索是在模型固定后、在验证集上做的后处理步骤，可以随时重新执行。因此优先保存 AUC 最优的参数是合理的。
-

五、实验设计与评估（用户编号为"六"）

Q15：为什么没有 XGBoost/Logistic Regression 基线？

确实没有。这是当前实验设计中最大的不足之一。

解释如下：本文的工作重心在于"MLP 在车险理赔预测中的模型结构设计与优化"，而非"多种模型在车险数据上的性能竞赛"。消融实验的设计逻辑是——以标准 MLP 为内部基线，逐一添加/移除组件（残差连接、LayerNorm、GELU、加权损失、自动阈值搜索等）来量化各组件贡献。这在方法论上是自洽的（验证的是"组件有效性"），但它不能回答"MLP 整体相对于其他模型是否更优"这个外部问题。

如果在后续工作中补做 XGBoost 和 Logistic Regression 的对比实验，无论结果是 MLP 更优、持平还是略差，都可以为论文增加实质性讨论：如果 MLP 更优，说明本模型的残差沙漏结构设计确实有效；如果持平或略差，则说明 MLP 在该任务上的主要价值在于架构灵活性和可扩展性而非绝对性能。

Q16：AUC 0.9604 是否过高？

AUC=0.9604 对于车险理赔预测任务来说是合理的，并非异常高：

- 正负比例约 1:4.37**，基准 AUC 为 0.5，0.96 说明模型的排序区分能力较强。
- 与文献对比**：Clemente et al. (2023) 在类似车险数据集上使用梯度提升树模型报告了 AUC 约 0.92-0.95。本模型的 0.9604 略高但仍在合理范围内。偏高的一个可能原因是：数据集中包含 `N_claims_history`（历史理赔次数）和 `R_claims_history`（历史出险率）这两个与目标变量强相关的历史特征，它们提供了较强的预测信号。
- 需要注意**：AUC 高不代表模型完美。测试集上 Claim 类的 Precision 为 0.7250，意味着在被预测为高风险的保单中约有 27.5% 实际并未理赔。Recall 为 0.8305，意味着约有 16.95% 的实际理赔保单被漏判。这些数值说明模型在实际部署时仍会存在不可忽视的误报和漏报。

Q17：验证损失低于训练损失的原因？

论文中的解释（训练时开启 Dropout 导致损失偏高）是主要原因，但不是唯一原因。完整分析：

- Dropout 效应（主因）**：训练损失是在每个 batch 的前向传播后即时计算的，此时 Dropout 处于激活状态，约 25% 的神经元被随机丢弃，模型的有效容量降低。验证损失是在整个 epoch 更新完成后、Dropout 关闭的状态下一次性计算的，模型使用了完整容量。因此验证损失的系统性偏低是预期行为。
- 训练损失的累积效应**：训练损失是每个 batch 损失的加权平均，而整个 epoch 的前半部分模型参数尚在更新中、损失较高；验证损失使用的是 epoch 结束后的最终参数。这也会使训练损失高于验证损失。
- 欠拟合可能性**：如果验证损失大幅低于训练损失，确实需要考虑欠拟合。但本模型的训练和验证准确率最终均收敛至约 90%，且曲线平滑无震荡，说明模型充分学习了数据中的判别模式，欠拟合的可能性较低。
- 数据分布差异**：训练集和验证集通过分层抽样保证了正负比例一致，理论上分布应当接近。但如果某些子群体在两个集合中的分布不均衡，也可能造成差异。
- 验证方法**：可以通过将 Dropout 比率临时设为 0（或极小的值，如 0.05）重新训练，观察训练损失和验证损失的相对位置是否恢复正常，来验证 Dropout 是否是唯一原因。模型还使用了标签平滑和权重衰减作为额外正则化手段——标签平滑使模型不会对训练标签过度自信，权重衰减持续抑制权重增长。这两种正则化在训练时全程生效，也会使训练损失略高于仅评估时的损失。

Q18: 训练损失 0.54 是否理想?

1. **朴素基线的损失值**: 对于正负比约 1:4.37 (正样本率约 18.6%) , 一个始终预测为负类 (全部输出 $p \approx 0$) 的模型的 BCE 损失约为 $-[0 * \log(p) + 0.814 * \log(1-p)]$, 当预测概率全为负类率 ($p=0.186$) 时, $BCE \approx -[0.186 * \log(0.186) + 0.814 * \log(0.814)] \approx 0.69$ 。全预测 0 时损失会更高 (因为正样本的损失 $-\log(\text{接近} 0)$ 趋近无穷大, 取极限为 $CE \approx +\infty$) 。一个合理的基线是输出恒为样本正类率 $p=0.186$, 此时 $BCE \approx -[0.186 * \ln(0.186) + 0.814 * \ln(0.814)] \approx 0.689$ 。
2. **模型损失 0.54 的提升**: 从 0.689 降至 0.54, 降幅约 21.6%。这说明模型确实学到了比"盲目猜测正类率"更好的预测能力。但由于类别不平衡和标签平滑的存在, 损失无法降到非常低的水平——标签平滑本身就为损失函数设定了一个下限 (即使完美拟合训练标签, 因标签平滑的存在损失也不会归零) 。
3. **为什么损失没有更低**: 除了标签平滑因素外, 车险理赔预测本身就是一个高噪声任务——两份在几乎所有可观测特征上都相似的保单, 可能因为未观测到的因素 (驾驶人的瞬时注意力、天气状况、道路突发情况等) 而导致完全不同的理赔结果。这种不可约误差决定了损失无法趋近于零。

六、模型可解释性 (用户编号为"七")

Q19: 特征独立性假设的局限性?

这是一个切中要害的问题。单特征局部消融的核心假设——"其他特征不变, 仅改变一个特征"——在特征相关时确实存在局限性。

具体问题: 如果车龄 (vehicle_age_years) 和车辆市场价值 (Value_vehicle) 高度负相关, 单独消融车龄时将其替换为训练集中位数, 但车辆价值仍保持原始值。这产生了一个现实中不太可能出现的"异常组合"——例如新车龄 (中位数) 配低车价 (原始值)。模型对这个异常组合的预测结果可能不能准确反映车龄的真正边际效应。

应对策略:

1. 当前方法的定位是"快速、直觉友好"的解释, 而非"严格因果归因"。它在实践中能给出大致方向正确的解释, 但不应被视为严格的特征重要性度量。
2. 论文中可以明确讨论这一假设的局限性, 并将其定位为"实用性解释工具"而非"严格的因果归因方法"。
3. SHAP (特别是 Kernel SHAP 或 Tree SHAP) 确实能更好地处理特征相关性, 因为它基于博弈论的 Shapley 值, 考虑了特征之间的交互。未来工作可以考虑替换为 SHAP。但 SHAP 的计算开销远大于单特征消融 (对于 28 个特征, 精确 SHAP 需要 2^{28} 次模型调用), 在实时推理场景下不太实用。一个折中方案是使用基于背景数据集的 Kernel SHAP 近似, 在可接受的计算时间内完成解释。

Q20: 基线值选择与"偏差效应"vs"特征重要性"?

基线值的具体选择:

- **连续数值变量**: 训练集中的中位数 (如年净保费 Premium 的中位数) 。
- **整数变量**: 训练集中的中位数取整 (如驾龄 seniority 的中位数取整) 。
- **日期类变量**: 训练集中该日期字段的中位数日期。
- **二值类别变量**: 训练集中的众数。
- **多值类别变量**: 训练集中的众数。

为什么是中位数而非均值：中位数对异常值更稳健。在保险数据中，保费和车价等字段存在极端值，均值会被拉偏，中位数更能代表"典型水平"。对于日期字段，中位数日期比均值日期更容易解释（"典型日期" vs "平均日期"）。

"偏差效应"vs"特征重要性"的区分：你说得对，当前方法衡量的是"该特征偏离典型水平所带来的概率变化"——即偏差效应。如果一个特征在所有样本中的取值始终接近中位数（方差很小），即使它客观上对预测很重要，也会因为消融前后概率变化不大而不会出现在解释列表中。这与特征重要性（如 permutation importance、SHAP 的全局重要性）度量的是不同的东西。

论文中确实没有明确做出这个区分，这是一个值得在答辩中主动承认并在修改论文时补充说明的点。

Q21：解释结果的稳定性？

连续两次对同一条样本请求预测，解释结果是否一致，取决于以下因素：

1. **模型推理是确定性的：**在 `model.eval()` + `torch.no_grad()` 模式下，对于相同的输入，模型输出完全确定。因此，解释方法中对每个特征的消融计算也是确定性的——连续两次请求同一条记录，解释结果完全一致。
2. **基线值是固定的：**基线值（中位数/众数）来自训练集统计量，存储在 `preprocess_reference.pkl` 中，不会随着推理请求而变化。
3. **唯一的非确定因素：**如果训练后重新拟合了 scaler 或 reference，基线值可能变化；如果切换了模型版本，模型参数变化会导致解释变化。这些都属于"有意的系统状态变更"而非"不稳定性"。

结论：在当前系统设计下，解释结果满足局部一致性。同一条样本多次预测的结果完全可复现。

七、系统架构与工程实现（用户编号为"八"）

Q22：模型版本管理机制？

系统具备基本的模型版本管理机制：

1. **训练产物绑定：**每次训练完成后，在训练任务的 `run_dir` 下保存以下文件：`best_model.pth`（模型权重）、`scaler.pkl`（标准化参数）、`preprocess_reference.pkl`（预处理参考信息，包含特征列名、填充值、截断上等）、`best_threshold.pt`（最优分类阈值）。这四个文件是绑定的——它们来自同一次训练，不能跨版本混用。
2. **保存到版本库：**管理员选择"保存权重"时，`TrainingManager.save_weights()` 将这些文件复制到 `outputs/saved_weights/` 目录，同时生成一个 JSON 元数据文件，记录文件路径、训练时间、性能指标和特征工程版本信息。
3. **加载时的兼容性校验：**推理服务加载模型版本时（`InferenceService._validate_version_metadata()`），会校验 scaler 的特征维度是否与 reference 一致，以及特征工程版本号是否匹配。如果不匹配，该版本会被标记为 `isCompatible: false` 并跳过。这确保了不会加载一个与当前推理 pipeline 不兼容的模型版本。
4. **回滚机制：**多个已保存的版本按照时间倒序排列在版本列表中。管理员可以切换默认模型版本到任意一个兼容的历史版本。但系统没有"一键回滚"按钮——管理员需要手动进入版本列表选择。
5. **不足之处：**缺少模型性能的自动对比（A/B test），没有模型部署的审批流程，且旧版本的清理策略未明确定义。

Q23: 权限控制在前端还是后端?

权限控制在**后端**实现，前端路由守卫是辅助性的 UI 层面的控制：

1. 后端权限控制（核心）：

- `SessionUserUtil.requireLogin(session)`：每个需要认证的接口都首先校验用户是否已登录。
- `SessionUserUtil.requireAdmin(session)`：管理员专属接口（如模型训练、用户管理）调用此方法，非管理员直接抛出异常。
- `SessionUserUtil.resolveDataScopeEmployeeNo(session)`：数据范围控制——管理员返回 `null`（可查全部数据），普通用户返回自己的工号。
- `SessionUserUtil.requireOwnerOrAdmin(creatorEmployeeNo, session)`：数据归属校验——普通用户只能操作自己创建的数据（`creator_employee_no` 匹配），管理员可操作所有数据。

2. 数据隔离的具体实现：以保单查询为例（`MotorInsuranceMapper.xml`），SQL 中动态拼接

`creator_employee_no` 条件——普通用户查询时自动过滤 `creator_employee_no = 当前用户工号`，管理员查询时不拼接此条件。

3. 前端路由守卫：`vue/src/router/index.js` 中通过 `meta.adminOnly` 标记管理员专属页面，导航守卫在路由跳转前检查用户角色。即使攻击者绕过前端直接调用后端 API，后端仍然会进行权限校验并拒绝越权请求。

4. 安全性结论：如果用户直接调用后端 API 接口并尝试越权访问其他用户的数据，后端会通过 `requireOwnerOrAdmin` 抛出“无权限查看或操作该数据”的异常。权限控制的防线在后端而非前端。

Q24: 数据库的外键约束与级联删除？

外键约束：数据库设计中，保单表（`motor_insurance`）、理赔记录表（`claim_record`）和车辆信息表（`vehicle_info`）通过 `ID` 字段（保单编号）进行逻辑关联。但这三个表之间的 `ID` 关联**没有通过数据库层面的 FOREIGN KEY 约束**来强制执行——关联逻辑由应用层代码维护。

级联删除：由于没有数据库层面的外键约束，自然也没有 `ON DELETE CASCADE` 机制。当用户删除保单时，Spring Boot 服务层代码需要在删除保单后手动删除对应的车辆信息和理赔记录。具体实现需要查看 `MotorInsuranceService.deleteById()` 方法。

引用完整性风险：缺少数据库层面的外键约束意味着：如果应用层代码有 bug 或手动操作数据库时出错，可能会出现“孤儿记录”——即车辆信息表或理赔记录表中存在 ID 但在保单表中没有对应记录。系统首页的“待办事项清单”（检查是否有保单缺理赔记录或缺车辆信息）能在一定程度上发现数据不完整的情况，但不能替代外键约束。

设计选择：在原型系统中省略外键约束是出于开发灵活性的考虑，但在生产环境中应当添加。

Q25: 异步训练与并发控制？

1. 训练是异步的：`TrainingManager.start_training()` 创建一个新线程

（`threading.Thread`），在后台运行 `run_training()`。方法本身立即返回 `job_id` 和初始状态，不阻塞调用方。

2. 进度获取：前端通过轮询 `/training/jobs/{job_id}` 接口获取训练进度。该接口返回 `job` 对象，其中包含 `currentEpoch`、`totalEpochs`、`history`（每个 epoch 的详细指标）、`status` 等字段。前端根据这些信息渲染进度条和训练曲线。

3. **并发控制**: `TrainingManager` 使用 `self._lock = threading.Lock()` 和 `self._active_job_id` 管理并发。在 `start_training()` 中, 如果已有运行的训练任务 (`status == "running"`), 会抛出 `RuntimeError("当前已有训练任务在运行, 请等待完成后重新启动新的训练")`。这意味着**同时只能有一个训练任务在运行**。
4. **中断恢复**: 服务重启时, `_load_latest_snapshot()` 会扫描 `training_jobs/` 目录下的 `job_state.json` 文件。如果发现状态为 `"running"` 的任务, 会将其标记为 `"failed"` 并附上错误信息"服务重启后, 进行中的训练任务无法自动恢复"。这意味着已训练的权重会丢失, 需要重新开始训练。
5. **正在训练中的预测请求**: 如果用户在训练过程中发起预测请求, 系统使用的是**旧的已加载模型**。训练过程中, 新模型的权重文件在训练任务的 `run_dir` 中逐步更新, 但只有当管理员显式执行"保存权重"操作后, 这些权重才会被复制到 `saved_weights/` 目录并可能被推理服务加载。在保存之前, 预测服务完全不受训练过程影响。

八、答辩高压追问 (用户编号为"九")

Q26: 如果 XGBoost 效果更好, MLP 还有什么价值?

即使 XGBoost 在原始性能指标上略优于或持平 MLP, 本工作的价值仍然存在:

1. **创新点不完全依赖"MLP 优于 XGBoost"的前提**。三个创新点——残差沙漏结构、自动阈值搜索、单特征消融解释——中, 只有第一个是 MLP 特有的。自动阈值搜索和可解释性方法是模型无关的, 同样可以用于 XGBoost。这部分工作构成了独立于模型选择的贡献。
2. **MLP 的可扩展性**: 如果后续需要引入车联网时序数据 (驾驶行为时间序列)、文本数据 (事故报告) 或图像数据 (定损照片), MLP 可以自然地扩展为多模态架构 (加入 LSTM/Transformer 分支处理时序, 加入 CNN 分支处理图像), 而 XGBoost 对非表格数据的处理能力有限。
3. **系统设计的完整性**: 即使模型部分的绝对性能提升有限, 整套系统——从数据管理、统计分析、模型训练、在线推理、可解释性分析到可视化的完整闭环——作为工程实践是有价值的。
4. **如果 XGBoost 显著优于 MLP** (例如 AUC 差距超过 0.02), 那就需要反思 MLP 在该任务上的合理性。但这是科学研究中的正常现象——否定结果同样有价值。

Q27: 论文最致命的问题是什么?

最致命的问题是: 没有与任何非深度学习基线模型进行横向对比实验。

论文在国内外研究现状中详细讨论了 GLM、XGBoost、梯度提升树等方法在车险预测中的应用, 但实验部分仅对比了 MLP 自身不同变体的性能。这使得论文无法回答"深度学习到底比传统方法好在哪里"这个核心问题。如果 MLP 的 AUC 比 XGBoost 低 0.01 但训练时间是 XGBoost 的 10 倍, 那么选择 MLP 的合理性就需要更充分的论证。

这个问题的根源在于: 论文将"MLP 的组件优化"作为实验设计的核心逻辑, 这在内部验证上是自洽的, 但在外部论证上是不完整的。补做 XGBoost 和 Logistic Regression 基线实验 (即使只用默认参数) 是修复这个问题最直接的方式。